

Vorlesung 2: Kondition, Stabilität & Zahldarstellung

§1 Numerische Aufgabe | §2 Konditionszahlen | §3 Stabilität & Algorithmen

1 Numerische Aufgabe: Grundbegriffe

Definition 2.1 — Numerische Aufgabe

Eine **numerische Aufgabe** berechnet endlich viele Ergebnisse $y_i, i = 1, \dots, n$, aus Eingabedaten $x_j, j = 1, \dots, m$, gemäß

$$y_i = f_i(x_1, \dots, x_m), \quad i = 1, \dots, n,$$

oder kompakt $y = f(x)$ mit $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$.

Beispiele:

- Lineares Gleichungssystem $Ay = b$: Eingabe $x = (A, b)$, Lösung $f(A, b) = A^{-1}b$.
- Quadratisches Polynom $y^2 - py + q = 0$: Eingabe $x = (p, q)$, Ausgabe $y = f(p, q)$.

Definition 2.1 — Absolute und relative Fehler

Bei fehlerhaften Eingabedaten $x_j + \delta x_j$ (z.B. Rundungs- oder Messfehler) ergeben sich fehlerhafte Resultate $y_i + \delta y_i$.

	Eingabe	Ausgabe
Absoluter Fehler	$ \delta x_j $	$ \delta y_i $
Relativer Fehler	$\frac{ \delta x_j }{ x_j }$	$\frac{ \delta y_i }{ y_i }$

Wichtig: Ob ein absoluter Fehler „groß“ ist, hängt vom Problem ab (z.B. 100 km: bei Himmelskörpern klein, bei Stadtentfernungen groß). Der relative Fehler ist oft aussagekräftiger.

2 Kondition: Empfindlichkeit des Problems

2.1 Differentielle Fehleranalyse und Konditionszahlen

Definition 2.2 — Relative Konditionszahl

Sei $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ zweimal stetig partiell differenzierbar und $|x_j|, |y_i| \neq 0$. Aus der Taylor-Entwicklung folgt:

$$\delta y_i = f_i(x + \delta x) - f_i(x) \approx \sum_{j=1}^m \frac{\partial f_i}{\partial x_j}(x) \delta x_j + \mathcal{O}(\|\delta x\|^2).$$

Die **relativen Konditionszahlen** von f im Punkt x sind:

$$\kappa_{ij}(x) = \frac{\partial f_i}{\partial x_j}(x) \cdot \frac{x_j}{y_i}$$

Sie messen, um welchen Faktor der relative Fehler in x_j den relativen Fehler in y_i verstärkt:

$$\frac{\delta y_i}{y_i} \approx \sum_j \kappa_{ij}(x) \frac{\delta x_j}{x_j}.$$

Terminologie 2.3 — Gut / Schlecht konditioniert

- $|\kappa_{ij}(x)| < 1$: **Fehlerdämpfung** — Ausgabefehler kleiner als Eingabefehler.
- $|\kappa_{ij}(x)| > 1$: **Fehlerverstärkung**.
- **Schlecht konditioniert**: falls $|\kappa_{ij}(x)| \gg 1$ für mindestens ein (i, j) .
- **Gut konditioniert**: andernfalls (alle $|\kappa_{ij}|$ moderat).

Die Konditionszahl ist eine **Eigenschaft des Problems**, nicht des Algorithmus.

2.2 Kondition der Addition und Multiplikation

Satz — Kondition elementarer Operationen

Addition $y = f(x_1, x_2) = x_1 + x_2$:

$$\kappa_1(x) = \frac{x_1}{x_1 + x_2}, \quad \kappa_2(x) = \frac{x_2}{x_1 + x_2}.$$

$\Rightarrow$ Addition ist **schlecht konditioniert**, wenn $x_1 \approx -x_2$ (d. h. $|x_1 + x_2| \ll |x_1|$).

Multiplikation $y = f(x_1, x_2) = x_1 \cdot x_2$:

$$\kappa_1(x) = \frac{x_1 \cdot x_2}{x_1 \cdot x_2} \cdot \frac{x_1}{x_1} = 1, \quad \kappa_2(x) = 1.$$

$\Rightarrow$ Multiplikation (und Division) sind **immer gut konditioniert** ($\kappa = 1$).

Achtung

Auslöschung: Die Subtraktion zweier fast gleicher Zahlen ist **schlecht konditioniert**. Dies führt zu starkem relativem Fehler im Ergebnis — und muss algorithmisch vermieden werden!

Beispiel 2.5 — Schlecht konditioniertes LGS

Das lineare Gleichungssystem

$$\begin{pmatrix} 1.2969 & 0.8648 \\ 0.2161 & 0.1441 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 0.8642 \\ 0.1440 \end{pmatrix}$$

hat die exakte Lösung $(y_1, y_2) = (2, -2)$. Eine Störung der rechten Seite um $\sim 10^{-4}$ führt zur gestörten Lösung $(0.9911, 0.4870)$ — relativer Fehler der Größe ~ 1 !

Ursache: $\det(A) \approx 10^{-5}$ — die Matrix ist nahezu singulär, daher riesige Konditionszahl.

3 Stabilität: Eigenschaft des Algorithmus

Terminologie 2.6 — Numerisches Verfahren und Stabilität

Ein **numerisches Verfahren** für $y = f(x)$ ist eine endliche Folge elementarer Operationen $\varphi^{(k)}$:

$$x \rightarrow \varphi^{(1)}(x) \rightarrow \varphi^{(2)}(\varphi^{(1)}(x)) \rightarrow \cdots \rightarrow \hat{y}.$$

Stabilität: Ein Algorithmus heißt **stabil**, wenn die akkumulierten Rechenfehler den unvermeidbaren Problemfehler (Kondition) nicht wesentlich übersteigen.

	Kondition	Stabilität
Was?	Eigenschaft des <i>Problems</i>	Eigenschaft des <i>Algorithmus</i>
Frage	Wie empfindlich ist f ?	Wie viel Fehler macht das Verfahren?
Kontrolle	Nicht wählbar	Durch Algorithmuswahl kontrollierbar

3.1 Fallstudie: Nullstellen quadratischer Gleichungen

Beispiel 2.8 — Quadratische Gleichung

Gegeben $p, q \in \mathbb{R}$, $q \neq 0$. Gesucht: Nullstellen von $y^2 - py + q = 0$.

Klassische Formel:

$$y_1 = \frac{p}{2} + \sqrt{\frac{p^2}{4} - q}, \quad y_2 = \frac{p}{2} - \sqrt{\frac{p^2}{4} - q}.$$

Vieta-Beziehungen: $y_1 + y_2 = p$, $y_1 \cdot y_2 = q$.

Kondition der Nullstellenberechnung: Die Konditionszahlen $\kappa_{ij}(p, q)$ können sehr groß sein, wenn $y_1 \approx y_2$ (d. h. die Nullstellen liegen nah beieinander). In diesem Fall ist die Aufgabe **schlecht konditioniert**.

Satz — Stabiler Algorithmus für $p < 0$, $0 \neq q \ll p^2/4$

In dieser Situation ist $|y_1/y_2| \ll 1$, also y_1 die betragsmäßig kleine Nullstelle. Stabiles Vorgehen:

Schritt 1: Berechne $w = \sqrt{p^2/4 - q}$ stabil (keine Auslöschung, da $q \ll p^2/4$).

Schritt 2: Berechne die *große* Wurzel direkt (kein Auslöschungsproblem):

$$\hat{y}_2 = \frac{p}{2} - w.$$

Schritt 3: Berechne die *kleine* Wurzel via Vieta — **nicht** aus der Lösungsformel:

$$\hat{y}_1 = \frac{q}{\hat{y}_2}.$$

Warum Variante A instabil ist: $\hat{y}_1 = p/2 + w$ leidet unter Auslöschung, da $p/2 \approx -w$ (beide fast gleich groß, verschiedenes Vorzeichen). Relativer Fehler $\sim 20\%$.

Warum Variante B stabil ist: $\hat{y}_1 = q/\hat{y}_2$ verwendet nur gut konditionierte Division. Relativer Fehler $\ll 1\%$.

Beispiel 2.10 — $p = 4$, $q = 0.01$, **Rundung auf 4 Stellen**

Berechnungen (gerundet): $\hat{u} = 4$ (exakt), $\hat{v} = 3.99$ (exakt), $\hat{w} = 1.9974\dots$, $\hat{y}_2 = 3.997$.
Exakte Lösung: $y_1 = 0.0025015645\dots$

	Ergebnis	Relativer Fehler
Variante A ($\hat{y}_1 = p/2 + w$)	0.003	$\approx 20\%$
Variante B ($\hat{y}_1 = q/\hat{y}_2$)	0.002502...	$\approx 0.017\%$

⇒ Variante B ist um Faktor ~ 1000 genauer!

Achtung

Merksatz: Bei quadratischen Gleichungen sollte **nicht** beide Wurzeln aus der klassischen Formel berechnet werden. Die betragsmäßig kleinere Wurzel immer via Vieta ($y_{\text{klein}} = q/y_{\text{groß}}$) bestimmen.

3.2 Python-Implementierung: Stabiler Algorithmus**Python-Code** — **Stabiler Quadratwurzel-Solver**

```
import math

def stable_quadratic(p, q):
    """Löst  $y^2 - py + q = 0$  stabil (für  $p < 0$ ,  $0 \neq q \ll p^2/4$ )."""
    discriminant = (p / 2)**2 - q
    w = math.sqrt(discriminant)
    y2 = p / 2 - w          # große Nullstelle (kein Auslöschungsproblem)
    y1 = q / y2             # kleine Nullstelle via Vieta (stabil!)
    return y1, y2

# Beispiel: p=4, q=0.01
y1, y2 = stable_quadratic(4, 0.01)
print(f"y1 = {y1:.10f}")    # ca. 0.0025015645
print(f"y2 = {y2:.10f}")    # ca. 3.9974984354
```

4 Vergleich: Kondition vs. Stabilität

Aspekt	Kondition	Stabilität
Definition	Empfindlichkeit des <i>Problems</i> f gegenüber Datenstörungen	Fehlerverhalten des <i>Algorithmus</i> relativ zur Kondition
Gehört zu	Problem / mathematischer Aufgabe	Numerischem Verfahren
Messbar durch	Konditionszahlen $\kappa_{ij}(x)$	Vergleich: berechneter vs. unvermeidbarer Fehler
Kontrollierbar?	Nein (gegeben durch Aufgabe)	Ja (Wahl des Algorithmus)
Beispiel gut	Mult./Division: $\kappa = 1$	Vieta-Variante B
Beispiel schlecht	Addition nahe-gleicher Zahlen	Klasische Formel für kleine Wurzel

Prüfungsscheckliste — Vorlesung 2

- ☐ Begriff „Numerische Aufgabe“ $y = f(x)$ erklären und zwei Beispiele nennen. → *Def. 2.1*
- ☐ Absoluten und relativen Fehler in Eingabe und Ausgabe definieren; erklären, warum relativer Fehler oft sinnvoller ist. → *Def. 2.1*
- ☐ Relative Konditionszahl $\kappa_{ij}(x) = \frac{\partial f_i}{\partial x_j} \cdot \frac{x_j}{y_i}$ herleiten (Taylor-Entwicklung). → *Def. 2.2*
- ☐ Konditionszahlen für Addition interpretieren: wann ist Addition schlecht konditioniert? → *Term. 2.3*
- ☐ Zeigen: Multiplikation und Division haben $\kappa = 1$ (immer gut konditioniert). → *§2*
- ☐ Begriff „Auslöschung“ erklären und ein Beispiel angeben, wo sie auftritt. → *Warnbox*
- ☐ Unterschied Kondition (Eigenschaft des Problems) vs. Stabilität (Eigenschaft des Algorithmus) erklären. → *Term. 2.6*
- ☐ Stabiler Algorithmus für quadratische Gleichung: Vorgehen in 3 Schritten skizzieren (große Wurzel direkt, kleine Wurzel via Vieta). → *Satz §3*
- ☐ Erklären, warum Variante A ($\hat{y}_1 = p/2 + w$) instabil ist (Auslöschung). → *Bsp. 2.8*
- ☐ Zahlenbeispiel $p = 4, q = 0.01$: relativen Fehler von Variante A (20%) und Variante B (0.017%) angeben und erklären. → *Bsp. 2.10*
- ☐ Konditionszahl des schlecht konditionierten LGS interpretieren: Störung $\sim 10^{-4}$ im Input → Fehler ~ 1 im Output. → *Bsp. 2.5*
- ☐ Python-Code für stabilen Nullstellen-Solver skizzieren (Hauptschritte). → *§3.1*